ZERON ACADEMY / AWS CLOUD ENGINEERING

SERVERLESS SOAP-TO-REST INTEGRATION GATEWAY

Architecture notes for students

BRIDGE A LEGACY SOAP CLIENT TO A MODERN REST API

A practical guide to request validation, XML/JSON transformation, Mustache templates, DynamoDB configuration, AWS serverless trade-offs, security, and high-traffic behavior.

SOAP/XML	Validate	Map	REST/JSON	SOAP/XML

What these notes emphasize

We can convert XML to JSON directly in Python. Mustache and DynamoDB are optional architectural choices used when transformation rules should be configurable separately from application code.

Project repository: https://github.com/sagarpanwarwork/soap-rest-gateway

Version 1.0 | October 2026 | Based on the Zeron Academy project code and architecture notes

Guide map

Use these notes as a companion to the repository and lab. They explain the implementation first, then evaluate whether each design choice is justified for production.

NO.	SECTION	PAGE
01	Scenario, scope and learning goals	p. 3
02	AWS architecture diagram	p. 4
03	End-to-end request lifecycle	p. 5
04	Contracts: SOAP/XML, Python, REST/JSON	p. 6
05	Mustache: purpose and limits	p. 7
06	Why store templates in DynamoDB?	p. 8
07	Lambda code walkthrough	p. 9
08	Faults, status codes and security	p. 10
09	Why serverless instead of EC2/ECS/EKS?	p. 11
10	Scaling model and capacity math	p. 12
11	What if traffic becomes huge?	p. 13
12	Optional buffered architecture	p. 14
13	Production readiness checklist	p. 15
14	Test plan and troubleshooting	p. 16
15	Classroom exercises	p. 17
16	Glossary and references	p. 18

A useful mental model

Separate format conversion from mapping configuration

Parsing XML and serializing JSON/XML are format operations. Choosing how fields from one contract map into another is a data-mapping decision. Mustache renders templates; DynamoDB stores them. Neither is mandatory for basic XML/JSON conversion.

Before the lab

- Read the project README and inspect src/lambda_function.py before changing infrastructure.
- Keep development and production state separate. Use the approved public egress IPv4 CIDR for WAF tests; never use 0.0.0.0/0 for this lab.
- DummyJSON is a mock service. POST /users/add simulates creation; it does not persist a real user.

01 | Scenario, scope and learning goals

Business problem

An enterprise has a legacy client that speaks SOAP 1.1/XML. A newer third-party service accepts REST/JSON. Rewriting the legacy client may be costly or impractical, so we introduce an adapter at the boundary: the client keeps its SOAP contract while the adapter translates to the downstream API contract.

Project objective

Accept one CreateUserRequest SOAP operation, validate it, map the extracted values into JSON, call a configured HTTPS endpoint, then transform the downstream JSON into a SOAP/XML response.

Scope of the implementation

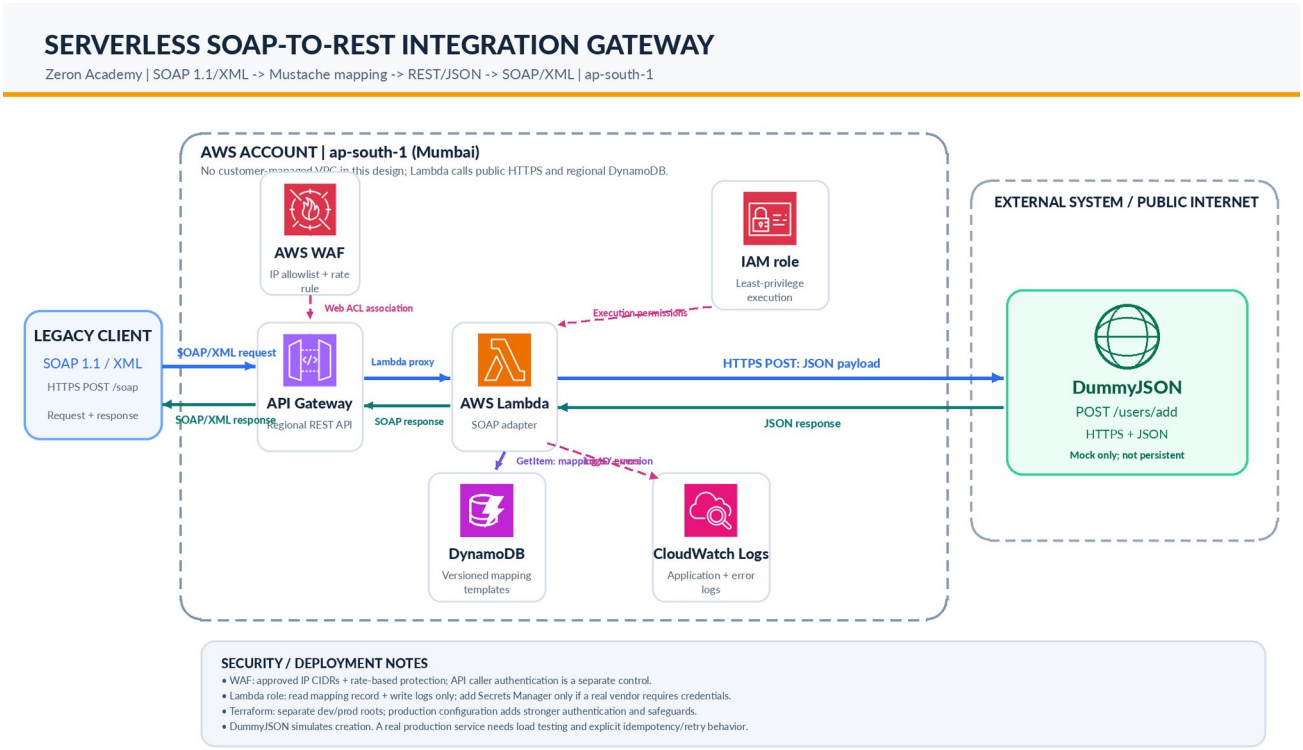
IN SCOPE	NOT CLAIMED BY THIS LAB
One SOAP 1.1 operation: CreateUserRequest.	A complete SOAP framework or support for arbitrary WSDL operations.
Fields: FirstName, LastName, Age; required/type/range validation.	A general-purpose schema engine for every external API contract.
DynamoDB record CreateUser#v1 with request/response Mustache templates.	Automatic mapping discovery or zero-code support for any new API.
Synchronous HTTPS call to DummyJSON; bounded response size and timeout.	Persistent account/user creation in a real business system.
Terraform-managed dev/prod environments; API Gateway REST API, WAF, Lambda, DynamoDB and CloudWatch.	Proof of production performance at high throughput without load testing.

After studying this guide, a student should be able to

- Trace a SOAP request through API Gateway, Lambda, DynamoDB, Mustache and a REST endpoint.
- Explain why XML parsing is not the same thing as mapping a source contract to a destination contract.
- Decide when templates belong in code, a deployment package, or a runtime configuration store.
- Estimate Lambda concurrency and identify API Gateway, DynamoDB and downstream bottlenecks.
- Compare Lambda with EC2 + ALB + Auto Scaling Groups, ECS/Fargate, and EKS using workload requirements.

02 | AWS architecture

The baseline design is synchronous: the client waits while Lambda calls the downstream REST endpoint and creates the SOAP response. The diagram uses AWS service icons from the supplied diagrams.net AWS architecture drawing; official icon resources are linked in the references.



Key boundaries: AWS WAF protects the API Gateway REST API stage; the Lambda execution role governs AWS permissions; the Lambda function has no customer-managed VPC in this design. API caller authorization is separate from the execution role.

03 | End-to-end request lifecycle



Stage-by-stage explanation

STAGE	WHAT HAPPENS	WHY IT MATTERS
1. Receive	API Gateway passes the HTTP request to Lambda using proxy integration.	The HTTP envelope and SOAP body arrive together; headers and base64 encoding must be handled.
2. Validate	Check content type, non-empty body, byte limit, SOAP 1.1 envelope, Body and supported operation.	Reject malformed, unexpected or oversized input before any outbound call.
3. Load mapping	Read CreateUser#v1 from DynamoDB using GetItem.	Keeps request and response templates outside the business orchestration code.
4. Render + call	Mustache renders JSON; Python parses it; urllib sends HTTPS POST.	Rendered text must be valid JSON and match the downstream contract.
5. Validate + map	Validate expected response fields, then render the SOAP response template.	A successful HTTP response can still contain invalid or unexpected JSON.
6. Return	Return HTTP status, SOAP body and response headers through API Gateway.	SOAP clients need a consistent response contract, including for failures.

Synchronous implication

The caller remains waiting while every step runs. The total response time includes API Gateway overhead, Lambda work, DynamoDB lookup, and downstream REST latency. A slow vendor can keep Lambda concurrency occupied even when the Python code is simple.

04 | Data contracts: SOAP/XML -> Python -> REST/JSON

Input contract (simplified)

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <CreateUserRequest xmlns="urn:zeron:users:v1">
      <FirstName>Ajmal</FirstName>
      <LastName>Student</LastName>
      <Age>29</Age>
    </CreateUserRequest>
  </soap:Body>
</soap:Envelope>
```

What the parser returns

```
user = {
    "firstName": "Ajmal",
    "lastName": "Student",
    "age": 29
}
```

Notice that XML element text is read as strings. The parser explicitly converts Age from text to an integer and checks the range 1 through 120. The dictionary is an intermediate representation; it is not automatically the final REST payload.

Downstream request contract (logical example)

```
{
  "firstName": "Ajmal",
  "lastName": "Student",
  "age": 29
}
```

Response direction

DummyJSON returns JSON containing fields such as id, firstName, lastName and age. Lambda validates the expected types, then the response template maps id to the SOAP response UserId element and wraps the fields in the expected SOAP envelope/namespace.

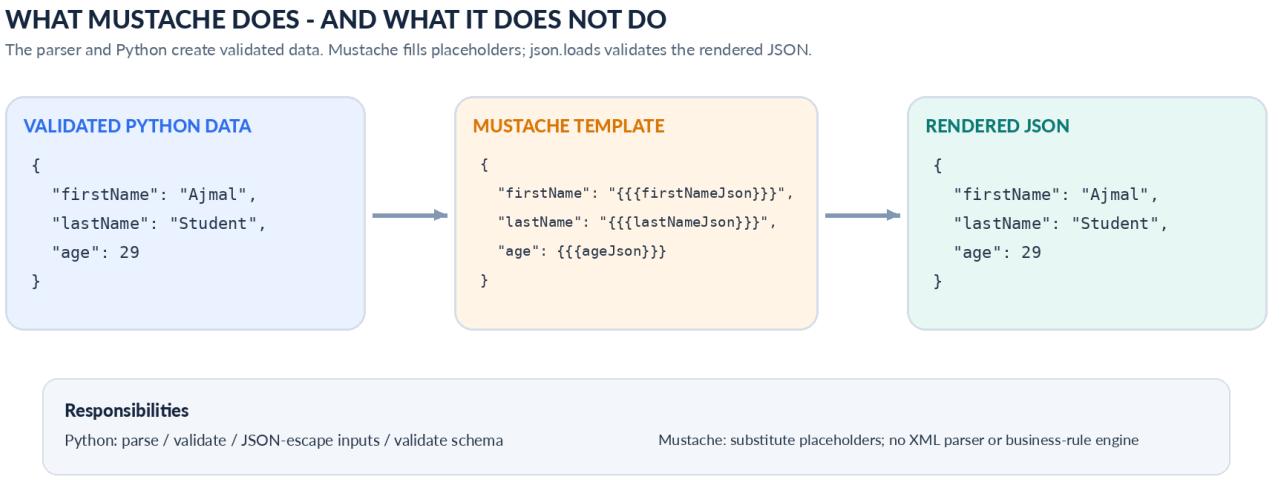
> **Contract validation is two-sided**
>
> Input validation protects the gateway from malformed or unsupported SOAP. Output validation protects the downstream service from invalid JSON and protects the legacy client from an unexpected downstream response.

Important SOAP details in the code

- SOAP 1.1 Envelope namespace is checked explicitly; the supported operation is CreateUserRequest only.
- _local_name() removes namespace URI/prefix details for field-name matching; this does not eliminate the need to validate the envelope namespace.
- _direct_child_text() requires exactly one direct child and rejects nested markup in simple fields.
- defusedxml is used for safer XML parsing; request byte size is bounded by MAX_XML_BYTES.

05 | Mustache: purpose, behavior and limits

Mustache is a template renderer. It substitutes placeholders in a predefined text structure with values supplied by the application. It does not parse SOAP, infer field mappings, validate business rules, or automatically convert XML to JSON.



The responsibilities stay separate

COMPONENT	RESPONSIBILITY
defusedxml / parser	Read XML safely, validate the envelope/operation, extract fields.
Python validation	Check required values, lengths, ranges and types.
Mustache	Insert supplied values into a prepared JSON or SOAP/XML template.
json.loads() + schema checks	Ensure rendered JSON parses and has the required fields/types.
SOAP response template	Wrap downstream fields in the expected SOAP response contract.

Why does the request template use triple braces?

Mustache normally HTML-escapes double-brace values. Triple braces render a value without Mustache HTML escaping. In this project, _json_string_contents() first JSON-escapes string content using json.dumps(), removes the surrounding quotes, and then inserts the pre-escaped content through triple braces. The final text is parsed with json.loads(). This is a deliberate technique for a JSON template, and it depends on maintaining the escaping and validation steps together.

Safety rule

Never put untrusted raw values into a JSON or XML template without context-appropriate escaping and post-render validation. HTML escaping, JSON escaping and XML escaping solve different problems.

When Mustache is a good fit

- Field renaming, rearranging fields, straightforward nesting, and rendering a known response envelope.
- Mappings are mostly structural and should be updated independently from orchestration code.
- Template changes can be versioned, validated, reviewed, tested and rolled back.

When Mustache is not enough

Complex conditionals, date conversion, calculations, array joins, cross-field business rules and schema evolution usually belong in explicit Python mapping functions or a more capable transformation system. Avoid encoding complicated business logic as increasingly fragile templates.

06 | Why store the templates in DynamoDB?

Start with the honest answer

We do not need DynamoDB to convert XML to JSON. We use it only if the mapping configuration has a lifecycle independent of Lambda code and must be retrieved/selected at runtime.

Three implementation options

OPTION	HOW IT WORKS	BEST FIT	TRADE-OFF
A. Direct Python mapping	Build dicts and XML elements in code.	One or few stable contracts; transformations are part of application logic.	Mapping changes require code change, tests and deployment.
B. Template packaged with Lambda	Ship Mustache files inside the deployment artifact.	Some separation of template from code, but releases are coordinated.	Template changes normally still require packaging/deployment.
C. Template in DynamoDB	Lambda reads a mapping record by ID/version at runtime.	Many integrations or frequent approved mapping changes independent of code.	More reads, latency, IAM and operational complexity; config can break requests.

Example mapping record

```
mapping_id:        "CreateUser#v1"
request_template:  { "firstName": "{{{firstNameJson}}}", ... }
response_template: <soap:Envelope> ... {{id}} ... </soap:Envelope>
```

What DynamoDB gives us

- Centralized mapping records keyed by operation and version.
- Runtime selection of the required mapping, if the application is built to route among mappings.
- Potentially change a compatible mapping without redeploying Lambda code.
- A place to manage multiple contract versions or tenant-specific mapping configuration, if requirements justify it.

What DynamoDB does not give us automatically

- It does not make the Lambda function generic; parser logic and contract/schema validation may still be hardcoded.
- It does not validate template correctness or guarantee safe deployment; build a validation/promotion/rollback process.
- It does not make changes risk-free. A bad active template can break requests immediately.

Specific to this implementation

_load_mapping() retrieves one configured record using a strongly consistent GetItem. The code then validates the rendered request against a fixed schema. Therefore, changing to a different JSON shape requires updating code/schema logic as well as the stored template.

Production recommendation

Keep DynamoDB when runtime configurability is a real requirement. Cache immutable/versioned mappings in the Lambda execution environment when appropriate, use a clear version/TTL strategy, and validate candidate templates before they become active. For one stable contract, direct Python mapping or a packaged template is often simpler.

07 | Lambda code walkthrough

The entry point is lambda_handler(event, context). API Gateway supplies request data in event; context includes invocation metadata such as aws_request_id. The function is organized into small helpers so each responsibility can be tested separately.

FUNCTION	WHAT IT DOES	TEACHING POINT
_response()	Returns statusCode, headers, body and isBase64Encoded for Lambda proxy integration.	One response shape for success and faults; Content-Type is text/xml.
_soap_fault()	Builds a SOAP 1.1 Fault with escaped text.	Expected failures become a protocol-compatible response, not raw exceptions.
_local_name()	Gets the XML local name without namespace URI/prefix details.	Useful for field matching, but envelope namespace still needs strict validation.
_direct_child_text()	Requires exactly one direct matching child; rejects child elements inside simple fields.	Avoids ambiguous values and unexpected nested markup.
_parse_soap_request()	Parses safe XML and validates the SOAP operation, fields and age range.	Turns external XML into validated Python data.
_load_mapping()	Reads mapping record from DynamoDB; fails if record/templates are missing.	Configuration is a dependency, so absence should fail visibly.
_json_string_contents()	JSON-escapes a string for the project's triple-brace template strategy.	Escaping must match the output format.
_render_request_payload()	Renders JSON with Mustache, parses with json.loads, checks fields/types.	Text rendering is not a substitute for structured validation.
_call_third_party()	POSTs HTTPS JSON with timeout, caps response size at 256 KiB and validates shape.	Bounded network calls prevent indefinite waits/oversized responses.
lambda_handler()	Controls the sequence and maps known failures to SOAP Faults.	Orchestration stays readable because parsing, mapping and calling are separated.

Environment variables

VARIABLE	PURPOSE
MAPPING_TABLE_NAME	Required DynamoDB mapping table name.
MAPPING_ID	Mapping key; defaults to CreateUser#v1.
THIRD_PARTY_API_URL	Required downstream HTTPS endpoint.
MAX_XML_BYTES	Maximum input XML bytes; default 65536 (64 KiB).
UPSTREAM_TIMEOUT_SECONDS	Downstream timeout; default 5 seconds.

08 | Faults, HTTP status codes and security

Observed response behavior in the code

HTTP STATUS	CONDITION	CLIENT-FACING BEHAVIOR
200	Valid request; downstream response validates; SOAP response renders.	Returns SOAP/XML success body.
415	Content-Type does not start with text/xml.	SOAP Fault: expected text/xml for this SOAP 1.1 lab.
400	Empty/invalid body, malformed XML, invalid fields, or unsupported operation.	General SOAP Client Fault.
413	XML request exceeds MAX_XML_BYTES.	SOAP Client Fault for oversized request.
504	Downstream call times out.	SOAP Server Fault: downstream service timed out.
502	Downstream HTTP error or connection failure.	SOAP Server Fault: downstream service failed/unreachable.
500	Unexpected processing exception.	Generic SOAP Server Fault; details stay out of the client response.

Security controls in the project

- AWS WAF: allow only configured IPv4 CIDRs and apply rate-based protection; default action is BLOCK.
- API Gateway REST API: chosen because REST APIs include AWS WAF integration and additional controls that HTTP APIs may not expose [2].
- Production API caller auth: the prod configuration can use AWS_IAM, requiring SigV4-signed requests and execute-api:Invoke permission. Dev may use NONE to simplify the classroom lab, but WAF restrictions still apply.
- Lambda execution role: least-privilege read access to the mapping table and logging permissions; it is not the same thing as client authentication.
- No customer-managed VPC in the current design. Public HTTPS egress works without attaching Lambda to a VPC; use a proper egress/NAT or endpoint design if networking requirements change.
- No raw request body in normal structured logs. If a real vendor needs credentials, store them in Secrets Manager; DummyJSON does not require a secret.

Important error-handling nuance

urllib commonly raises HTTPError for upstream HTTP 4xx/5xx responses. In this code these are caught and mapped to 502. The success-status check after urlopen is defensive but usually redundant for standard urllib behavior.

09 | Why serverless instead of EC2 + ALB + Auto Scaling Groups, or EKS?

The choice is about operational fit, traffic shape, control requirements and cost—not about one service being universally superior. This request/response adapter is short-lived and mostly network-bound, making Lambda a sensible baseline for the current project.

FACTOR	API GATEWAY + LAMBDA	EC2 + ALB + ASG	ECS / FARGATE	EKS
Infrastructure ownership	AWS manages execution environments and scale mechanics.	Team manages instances, OS patching, AMIs, ASG and target registration.	Team manages task/service definitions and scaling; Fargate reduces node operations.	Team owns Kubernetes operations, add-ons, node/capacity strategy and upgrades.
Scaling model	Per-request concurrency expands within quotas; no server fleet sizing.	ASG adds/removes instances based on metrics/desired capacity; scale-out takes time.	Service Auto Scaling adds/removes tasks; configure min/max and capacity.	HPA/KEDA scales pods; Cluster Autoscaler/Karpenter or Fargate handles capacity, depending setup.
Idle/steady cost	Low idle cost for sporadic traffic; cost grows with requests and duration.	Instances and ALB baseline cost continue while capacity is provisioned.	Fargate tasks and ALB cost while tasks run; minimum task count is a design choice.	Cluster/control-plane and compute costs plus meaningful ops overhead.
Best-fit signals	Short, stateless request/response work with variable traffic.	Long-running processes, OS/agent control, predictable sustained utilization.	Container workloads, long-lived services, custom runtime or steady traffic.	Established Kubernetes platform needs, ecosystem, multi-service orchestration.
Operational trade-off	Quotas, cold-start/initialization effects, duration limits, concurrency and vendor bottlenecks.	Patch, scale, capacity, health, deployment, networking and failure handling.	Task sizing, autoscaling policies, image management, deployment and ALB.	Highest operational complexity; needs a real reason beyond “high traffic”.

Why ALB and Auto Scaling Groups are not needed by default here

API Gateway already supplies an HTTPS API front door, request controls and Lambda integration. Lambda supplies on-demand execution. Adding EC2 instances, an ALB and an ASG would introduce server fleet sizing, health checks, instance patching and deployment lifecycle without proving a benefit for this short-lived adapter.

When the decision could change

- Traffic becomes high and sustained, and measured Lambda/API Gateway cost or latency is worse than a container/instance-based alternative.
- The integration requires long-lived connections, custom OS agents, specialized binaries, or workloads outside the Lambda request model.
- An organization already operates a well-managed ECS/EKS platform and the adapter fits that platform better.

Decision rule

Start with the simplest architecture that meets latency, throughput, security and cost goals. Load-test before moving to EC2 or EKS just because traffic is expected to grow.

10 | Scaling model and capacity math

1. Estimate Lambda concurrency

```
Concurrency ~= requests per second (RPS) x average duration in seconds
Estimated max RPS ~= available concurrency / average duration
```

Example: at 300 RPS and 0.6 seconds average Lambda duration, estimated concurrency is 300 x 0.6 = 180 concurrent executions. If the same traffic starts taking 2 seconds because the vendor slows down, concurrency becomes 300 x 2 = 600. Slower dependencies consume more concurrency even when RPS is unchanged.

> **This is a sizing estimate, not a guarantee**
>
> Concurrency math assumes a reasonably stable arrival rate and average duration. Tail latency, burstiness, retries, other functions in the account, timeouts and downstream throttles change the actual result.

2. Know the service boundaries

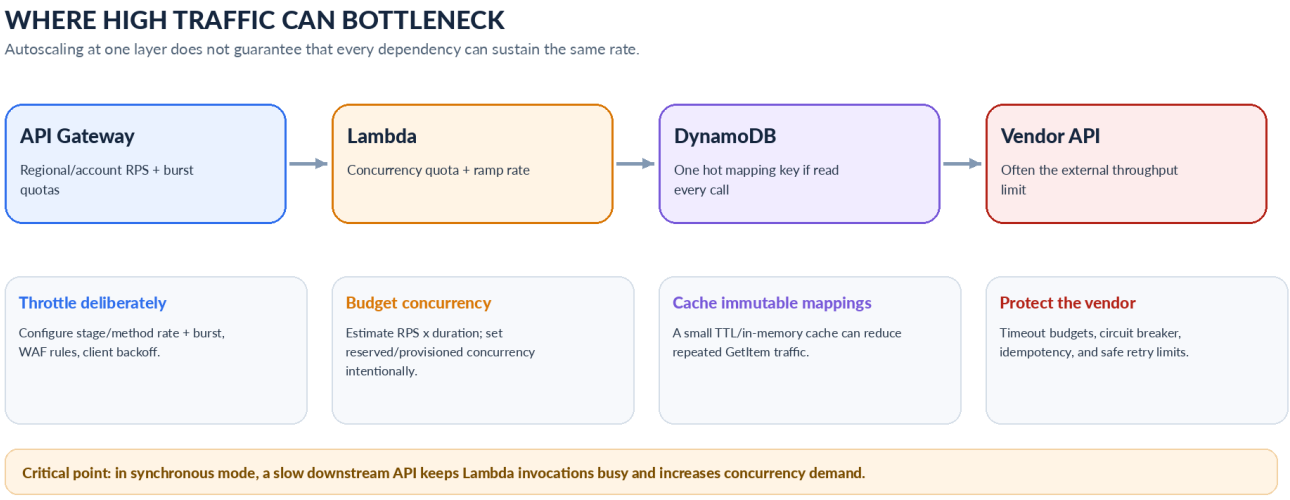
LAYER	WHAT TO CHECK	EFFECT WHEN EXCEEDED
API Gateway REST API	Account/Region throttle quota, stage/method rate and burst targets. AWS documentation lists a default account-level throttle quota of 10,000 RPS and bucket capacity of 5,000 in most Regions; actual quotas and exceptions must be checked [3].	Throttles requests; clients can receive HTTP 429. API Gateway limits are best-effort targets, not guaranteed ceilings [4].
Lambda	Account concurrency, reserved concurrency, provisioned concurrency and function scaling behavior. AWS documentation currently describes scaling up to 1,000 additional execution environments per function every 10 seconds per Region; check current quota docs [5][6].	Invocations may be throttled if concurrency is exhausted or traffic ramps faster than scaling.
DynamoDB	Capacity mode, item size, table quotas and access pattern. On-demand helps handle variable demand but table/partition constraints still apply [8][9].	GetItem latency/throttling can increase. Re-reading one fixed mapping key at high rate can create a hot-key access pattern.
External REST API	Published vendor rate limit, connection limits, per-customer quotas, latency and availability.	Lambda waits longer, increases concurrency pressure and may return 502/504 failures.
CloudWatch / cost	Log volume, retention, alarms and cost budgets.	High-volume logs and retries can materially increase cost; missing alarms delay response.

3. Reserve and pre-warm intentionally

- Reserved concurrency sets a ceiling and reserves account capacity for a function; it can protect the vendor and reserve capacity for critical work, but too low a value throttles traffic.
- Provisioned concurrency initializes environments in advance and can help predictable bursts or cold-start-sensitive latency; it adds cost and does not remove API Gateway, account concurrency or downstream limits.
- Reduce duration before buying more concurrency: profile dependency calls, adjust Lambda memory/CPU, eliminate unnecessary work, and define tight timeouts.

11 | What if the architecture receives huge traffic?

“Huge traffic” is not a single capacity number. The correct design depends on target RPS, burst shape, p95/p99 latency, payload size, vendor limits, failure tolerance and whether the client must receive the final SOAP result in the same request.



First response: keep the synchronous contract

1. Measure the baseline: steady RPS, burst RPS, p50/p95/p99 latency, Lambda duration/concurrency/throttles, API Gateway 4xx/5xx/429s, DynamoDB consumed/read-throttle metrics, and vendor latency/error rates.
2. Check and request quotas: API Gateway account/Region throttle limits, Lambda concurrency, and downstream vendor quotas. Do not assume service quotas are unlimited or identical in every account/Region.
3. Set deliberate back-pressure: API Gateway stage/method throttles, WAF rate controls, client retry with exponential backoff and jitter, and vendor-aligned Lambda reserved concurrency.
4. Reduce unnecessary work: cache read-only versioned templates in warm Lambda environments with a TTL or immutable version strategy; avoid a DynamoDB read for every request if runtime update semantics do not require it.
5. Protect the synchronous chain: configure a total latency budget, bound network timeouts, use circuit-breaker/fail-fast behavior where appropriate, and never retry a non-idempotent write blindly.

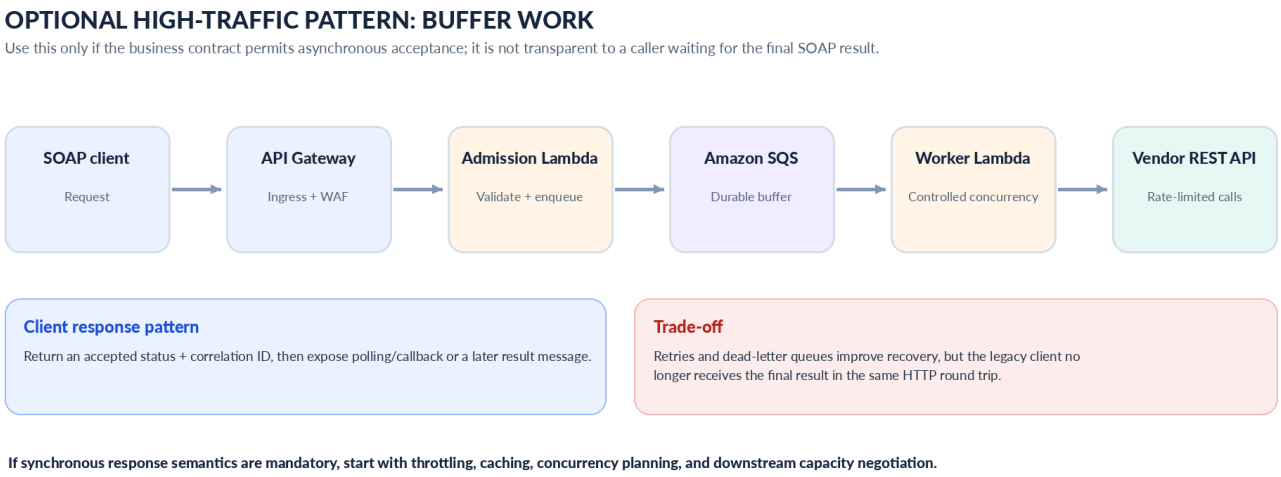
Then decide if the architecture needs a different compute or interaction model

TRAFFIC / WORKLOAD SIGNAL	POSSIBLE RESPONSE
Unpredictable bursts, short work	Keep API Gateway + Lambda; verify quotas, configure concurrency, cache mapping and load-test bursts.
Predictable bursts with strict latency goal	Consider provisioned concurrency, pre-warming, realistic load tests and capacity/quotas planning.
Sustained high throughput with stable traffic	Compare Lambda economics and latency with ECS/Fargate or EC2 + ALB/ASG using measured numbers.
Vendor API is the bottleneck	Negotiate vendor limits, throttle worker/request concurrency, or change the interaction contract; more Lambda will not make the vendor faster.
Work can complete asynchronously	Buffer accepted work with SQS and process through controlled-concurrency workers; add retries, DLQ, idempotency and status retrieval.
Large/high-frequency mapping reads	Cache immutable templates or re-evaluate the storage/access pattern; verify DynamoDB hot key, capacity mode and quotas.

Avoid this mistake

Do not increase Lambda concurrency without first understanding the downstream API limit. More parallel Lambda calls can overload the vendor faster and create a retry storm.

12 | Optional high-traffic pattern: buffer work



When a queue helps

Amazon SQS can decouple request acceptance from downstream execution. A worker Lambda can use controlled concurrency to respect a vendor rate limit. A queue absorbs short bursts, retries transient failures, and supports a dead-letter queue (DLQ) for messages that cannot be processed.

What changes for the SOAP client?

- The simple synchronous contract returns the final SOAP result after the downstream call completes.
- An asynchronous pattern generally returns an acceptance result and correlation ID, then exposes polling, callback, or another result delivery mechanism.
- If the existing SOAP client requires the final UserId in the same HTTP response, a queue is not a drop-in performance switch. It changes client-visible semantics, or requires a carefully designed synchronous wait pattern that can undermine the benefit of buffering.

Retries and idempotency

If a write operation times out, the gateway may not know whether the downstream service completed the operation. Retrying can create duplicates unless the downstream API supports an idempotency key or the adapter can safely deduplicate. In an SQS design, message retries and DLQ handling must be designed explicitly.

A queue is not a magic scaling layer

SQS smooths arrival rate and separates lifecycles, but does not increase the vendor's permitted throughput. Backlog age and queue depth must be monitored; workers need a concurrency limit aligned with the downstream contract.

13 | Production-readiness checklist

Application and contracts

- Validate SOAP envelope namespace, operation, fields, length and numeric ranges; confirm whether SOAPAction or WSDL-specific rules are required by the real client.
- Validate the downstream JSON schema and the generated SOAP response; add representative/edge-case fixtures.
- Define clear HTTP status to SOAP Fault mapping, request IDs/correlation IDs and safe error messages.
- Define idempotency and retry behavior for write operations; set a total timeout budget across API Gateway, Lambda and the vendor.
- Pin dependencies, test Lambda packaging, and add automated unit/integration tests.

Security and deployment

- Use an appropriate API caller authentication/authorization model. WAF IP allowlisting is an extra layer, not identity authentication.
- Use least-privilege IAM; separate client invoke permissions from Lambda execution permissions.
- Store vendor credentials in Secrets Manager only when required; never expose secrets in Terraform variables, templates or logs.
- Keep dev and prod state/configuration separate; review terraform plan before apply; test rollback and template promotion.
- Keep WAF default-block/allowlist CIDRs current; avoid opening the endpoint to 0.0.0.0/0 for convenience.

Scale and operations

- Load-test expected steady RPS, burst RPS, slow vendor responses, payload sizes, and error behavior in a controlled environment.
- Configure API Gateway stage/method throttles, Lambda concurrency limits, alarms and service-quota review.
- Choose DynamoDB on-demand vs provisioned capacity based on access pattern and cost; cache immutable mapping versions when safe.
- Monitor latency percentiles, 429/5xx rates, Lambda throttles/concurrency/duration/errors, DynamoDB throttle metrics, vendor failures, queue depth if used, and cost budgets.
- Use a documented runbook for vendor outage, throttling, configuration rollback and credential rotation.

What “production-oriented” means in this repository

The repository provides stronger production-oriented settings for the prod root (for example, AWS_IAM authorization, longer log retention, DynamoDB point-in-time recovery and deletion protection). It is still a training integration while its downstream dependency is DummyJSON and while performance, operational, and business requirements have not been proven with real workload tests.

14 | Test plan and troubleshooting

Minimum functional and negative tests

TEST	EXPECTED RESULT	WHAT IT PROVES
Valid CreateUserRequest	HTTP 200 with SOAP/XML response.	End-to-end mapping and outbound call work.
Content-Type application/json	HTTP 415 + SOAP Client Fault.	Request content type is enforced.
Malformed XML / missing SOAP Body	HTTP 400 + SOAP Client Fault.	Parser/contract validation rejects invalid input.
Missing FirstName / LastName / Age	HTTP 400 + SOAP Client Fault.	Required fields are validated.
Age non-integer / outside 1..120	HTTP 400 + SOAP Client Fault.	Type conversion and range checks work.
Request body larger than limit	HTTP 413 + SOAP Client Fault.	Request size is bounded.
Missing mapping record / invalid template	HTTP 500 + generic SOAP Server Fault; details in logs.	Configuration dependency fails visibly.
Slow/unreachable vendor	HTTP 504 on timeout or 502 on connection/vendor HTTP error, as mapped.	Downstream failures are bounded and translated.
Unapproved client IP	WAF blocks before Lambda invocation.	Edge access rules are active.
Prod call without valid SigV4 credentials	API Gateway rejects request.	API caller authorization is not confused with Lambda IAM role.

Troubleshooting map

OBSERVATION	LIKELY CAUSE	NEXT CHECK
HTTP 403 before Lambda	WAF CIDR does not match client egress IP or IAM authorization failed.	Confirm public egress IPv4; inspect WAF logs and SigV4 signing/execute-api:Invoke permissions.
HTTP 429	API Gateway throttle or Lambda concurrency throttle.	Inspect API Gateway metrics, Lambda Throttles/ConcurrentExecutions and quota settings.
HTTP 502 / 504	Vendor HTTP/connection error or timeout.	Check downstream status/latency, URL, timeout, DNS/network egress and vendor quota.
HTTP 500	Missing mapping, rendered-contract failure, unexpected response or another exception.	Use requestId to correlate CloudWatch logs; do not log raw SOAP contents.
High latency with low Lambda CPU	Synchronous network dependency is slow.	Measure vendor and DynamoDB latency separately; adding memory may not fix the dependency.
DynamoDB throttling at high RPS	Capacity constraint or concentrated hot-key access.	Check ConsumedReadCapacityUnits/ThrottledRequests, mode, key access pattern and mapping cache.

15 | Classroom exercises and discussion prompts

Exercise A - trace a valid request

Using the sample request, write down the exact value and type of each field at three points: extracted XML text, the returned Python dictionary, and the rendered outbound JSON. Explain why Age changes from text to an integer.

Exercise B - change the downstream schema

Assume the vendor changes firstName/lastName to given_name/family_name and nests both under user. Identify every part of this implementation that must be changed: mapping record, hardcoded schema validation, response expectations, tests, and possibly the response template.

Exercise C - defend the DynamoDB decision

Argue both sides: why runtime templates could help a gateway with 40 integrations, and why one stable CreateUser operation may be simpler with direct Python mapping or a packaged template. Include operational risks, caching and rollback.

Exercise D - capacity sizing

The gateway receives 450 requests per second. Average end-to-end Lambda duration is 0.8 seconds. Estimate concurrency. Then estimate the effect if vendor latency increases average duration to 2.4 seconds.

> **Expected arithmetic**
>
> At 450 RPS x 0.8 seconds = 360 concurrent executions. At 450 RPS x 2.4 seconds = 1,080 concurrent executions. The second estimate exceeds the common default regional Lambda account concurrency quota of 1,000 before considering other functions; check the actual quota and dependencies.

Exercise E - preserve synchronous semantics

A downstream service can only handle 120 requests per second, but incoming traffic peaks at 600 RPS. The SOAP client requires the final result in the same response. Propose a safe design. Explain why simply increasing Lambda concurrency may worsen the problem and why SQS changes the contract.

Quick knowledge check

QUESTION	EXPECTED POINT
Does Mustache convert XML to JSON by itself?	No. Python parses/validates XML; Mustache renders a supplied template.
Is DynamoDB mandatory for the converter?	No. It is useful only when runtime template storage is justified.
Does Lambda auto-scaling mean unlimited traffic?	No. API Gateway, Lambda quotas, DynamoDB, vendor limits and latency all constrain throughput.
Does WAF IP allowlisting authenticate a user?	No. It limits source network addresses; identity authorization is separate.
When does a queue help?	When work can be accepted asynchronously and downstream processing can be decoupled/rate-limited.

16 | Glossary and official references

TERM	MEANING IN THIS PROJECT
SOAP 1.1	XML-based protocol/message envelope used by the legacy client.
REST/JSON	Downstream HTTP API contract; this project sends JSON over HTTPS POST.
Adapter / gateway	Translation and orchestration layer between different API contracts.
Mustache	Logic-less template renderer that replaces placeholders with supplied values.
Mapping template	A template defining how validated fields are arranged into a target request or response.
DynamoDB GetItem	Read operation used to retrieve the configured mapping record by mapping_id.
Lambda concurrency	Number of invocations running at the same time.
Reserved concurrency	Function-level reservation and upper bound on concurrent executions.
Provisioned concurrency	Pre-initialized Lambda execution environments for lower initialization latency.
Back-pressure	Controls that prevent an upstream layer from overwhelming a downstream dependency.
Idempotency	Repeated execution of the same logical request does not cause unintended duplicate effects.
DLQ	Dead-letter queue; stores messages that could not be processed successfully after retry policy.

References for verification and further reading

[1] AWS Architecture Icons https://aws.amazon.com/architecture/icons/ - Official AWS-approved icon package and usage information.

[2] Choose between REST APIs and HTTP APIs https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-vs-rest.html - Feature differences, including WAF integration.

[3] Amazon API Gateway quotas https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html - Account/Region request throttling quotas and burst capacity.

[4] Throttle REST APIs https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-request-throttling.html - Best-effort rate/burst throttling and 429 behavior.

[5] Lambda concurrency https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html - Concurrency model and concurrency quota concepts.

[6] Lambda scaling behavior https://docs.aws.amazon.com/lambda/latest/dg/scaling-behavior.html - How Lambda scales and rate-of-scale limits.

[7] Lambda best practices https://docs.aws.amazon.com/lambda/latest/dg/best-practices.html - Timeouts, retries, concurrency controls and downstream capacity.

[8] DynamoDB on-demand capacity https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/on-demand-capacity-mode.html - Scaling characteristics and throughput considerations.

[9] DynamoDB partition-key design https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html - Access patterns, partition distribution and hot-key risk.

[10] Lambda VPC configuration https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc.html - Networking behavior when attaching a Lambda function to a VPC.

[11] SQS best practices https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-best-practices.html - Queue-based decoupling and message processing patterns.

[12] Project repository https://github.com/sagarpanwarwork/soap-rest-gateway - Source code, Terraform, tests, deployment workflow and architecture notes.

Quota note: the AWS quota figures in these notes are documented defaults current at the time of preparation, not guaranteed capacity. Actual quotas, regional exceptions and approved increases should be verified in the AWS account/Region used for the lab.